

Problem 1

Algorithm 1 Min Cost

Require: Two Strings X, Y s.t. $|X| \leq |Y|$

Ensure: Finds the min length of combining the two strings, so they are both subs sequences in order in our new string.

```

1: function MINCOST( $X[0, \dots, n-1], Y[0, \dots, m-1]$ )
2:   if  $|X| = 0$  then
3:     return  $|Y|$ 
4:   if  $|Y| = 0$  then
5:     return  $|X|$ 
6:    $prev[0, \dots, n]$ 
7:    $curr[0, \dots, n]$ 
8:   for  $j \in \{0, \dots, n\}$  do
9:      $prev[j] \leftarrow j$ 
10:  for  $i \in \{1, \dots, m\}$  do
11:     $curr[0] \leftarrow i$ 
12:    for  $j \in \{1, \dots, n\}$  do
13:      if  $X[i-1] = Y[j-1]$  then
14:         $curr[j] \leftarrow 1 + prev[j-1]$ 
15:      else
16:         $curr[j] \leftarrow 1 + \min(prev[j], curr[j-1])$ 
17:     $prev \leftarrow curr$ 
18:  return  $P[n]$ 

```

For the last extra credit problem I said to return $P[n]$ when I should have returned $curr[n]$.

Problem 2

(a) 5,6,9,15,18

Total Weight:53

- (b) The proof is flawed because it mistakes the determinism of Kruskal's algorithm for a proof of the MST's uniqueness. Just because the algorithm always produces the same tree doesn't rule out the possibility that another spanning tree with the same total weight exists. A correct argument must demonstrate that any other spanning tree would have a greater total weight, not merely that one algorithm always picks the same edges.
- (c) Assume for contradiction that there are two distinct minimal spanning trees T_1, T_2 on a graph with unique edge weights.

Let e be the min edge that appears in one tree but not the other. WLOG let e be in T_1 .

Thus, in T_2 if we add e to it there will be a cycle call this C . Since T_1 does not have a cycle this means that C has an edge f that is in T_2 , but not T_1 , since the edge weights are unique we get that $w(e) < w(f)$. This leads to a contradiction since we can take f out of T_2 then add e . This leads to a contradiction because we create a spanning tree with less weight than the original T_2 .

Therefore, if a graph has unique edge weights it has a unique MST.

Problem 3

Algorithm 2 Min Refills

(a) **Require:** R is the distance she can go without refill, D array of distances of trails.

Ensure: Finds min number of times she needs to stop and refill water.

```

1: function MINREFILLS( $R, D$ )
2:    $refill \leftarrow 0$ 
3:    $currWater \leftarrow R$ 
4:   for  $i \in \{1, \dots, n\}$  do
5:     if  $currWater < D[i]$  then
6:        $refills \leftarrow refills + 1$ 
7:        $currWater \leftarrow R$ 
8:        $currWater \leftarrow currWater - D[i]$ 
9:   return  $refills$ 

```

This runs in $O(n)$ time due to just looping n times on $O(1)$ operations.

(b) Let $Alg = \{i_1, i_2, \dots, i_k\}$ be the set of trail indices chosen by the algorithm, and let Opt be an optimal solution. If they are the same we are done. If not let i_{diff} be the first index they differ.

Thus

$$Alg = \{i_1, i_2, \dots, i_{diff-1}, i_{diff}, i_{diff+1}, \dots, i_k\}$$

$$Opt = \{i_1, i_2, \dots, i_{diff-1}, j, \dots\}$$

Now consider Opt' s.t. we switch out j for i_{diff}

This is still optimal because the algorithm selects i_{diff} because it is the last trail reachable with the current water supply before running out. This means that refilling at i_{diff} guarantees that the segment from the previous refill to i_{diff} and then from i_{diff} onwards remains within the allowed distance R . Replacing j with i_{diff} in Opt does not create any gap that would violate the constraints because i_{diff} is the last possible trail to take water from so $j \leq i_{diff}$.

Thus Opt' and OPT have the same number of refills thus Opt' is an optimal solution.

This guarantees that our algorithm is correct because we can keep repeating this process on Opt' and get a new solution that agrees on more variables, we then keep repeating till the Opt' we are creating becomes Alg .

(c) This is false here is a counter example $R = 4$, $D = [1, 2, 2]$, $C = [1, 1, 100]$. In the greedy algorithm we will go as far as possible, so she will do trail 1, 2 then stop on 3 and pay

the 100. However, there is a better way do trail 1 then refill at trail 2 then do trail 2,3. This will cost 1 dollar.

In the original problem (minimizing the number of refills) our exchange/induction proof showed that delaying a refill until forced is optimal. That proof depended solely on the distances and the fact that you want to minimize stops. When refill costs differ, however, “waiting as long as possible” might force you to refill at a very expensive station (as in our counterexample, where the greedy rule made you refill at trail 3 for \$100). Thus, the exchange argument breaks down because you can sometimes reduce the total cost by taking an extra refill at a cheaper station, even if it is not the “last possible stop.”

Problem 4

- (a) Every 0-1 knapsack solution is also a feasible solution to the fractional knapsack problem, so the fractional version's optimal value cannot be lower than that of the 0-1 version. Moreover, because the fractional knapsack allows you to take portions of items to exactly fill the knapsack's capacity (which you wouldn't be able to in 0-1 knapsack), it can only achieve a higher or equal total value.
- (b) Consider $V = [1, 1, 20, 30, 100]$ is the values and $W = [1, 1, 4, 2, 50]$ is the weights, and with a capacity of 1. The value greedy algorithm will only get a value of 2 since it will put $100/50$ into the sack and be done. The weight greedy algorithm will give a value of 1 since it will just put the smallest weight of 1 into the sack. However, these are both not optimal here is a better solution take $1/2$ of the 30 item thus we get a value of 15.
- (c) WLOG suppose that the items are in sorted order by relative value, from largest to smallest. Let $Alg = p_1, p_2, \dots, p_n$ be the fractions of items $1, 2, \dots, n$ chosen. Let $Opt = q_1, q_2, \dots, q_n$ be an optimal solution. If $Alg = Opt$ then we are done. If not consider the smallest index j s.t. they are different.

Since Alg is greedy and takes the most fraction it can take from the highest relative value to lowest we know that $p_j > q_j$ since the solutions are not equal.

Define the extra weight Alg took as

$$\Delta = (p_j - q_j)w_j$$

WLOG assume that the knapsacks are full since if they are not, and we still have an Opt solution that means we could fit all objects in the sack. Thus, since both solutions fill the knapsack we must have taken more weight from later index in Opt to fill in the δ we didn't take from item j .

Let $S = \{i > j \mid q_i > p_i\}$ be the set of extra weight we took to compensate.

Thus,

$$\sum_{i \in S} (q_i - p_i)w_i = \Delta$$

Now for Opt' for item j let $q'_j = p_j$ then for each $i \in S$ let $q'_i = p_i$, and leave the other fractions unchanged.

Thus overall we have not changed the weight of the new Opt' solution we just shifted where we took the weight from.

Thus the change of value of Opt to Opt' is

$$\Delta \frac{v_j}{w_j} - \sum_{i \in S} (q_i - p_i) v_i = \Delta \frac{v_j}{w_j} - \sum_{i \in S} (q_i - p_i) \frac{v_i w_i}{w_i}$$

Since for each $i \in S$ we have $v_i/w_i \leq v_j/w_j$

$$\Delta \frac{v_j}{w_j} - \sum_{i \in S} (q_i - p_i) \frac{v_i w_i}{w_i} \geq \Delta \frac{v_j}{w_j} - \sum_{i \in S} (q_i - p_i) \frac{v_j w_i}{w_j} = \Delta \frac{v_j}{w_j} - \Delta \frac{v_j}{w_j} = 0$$

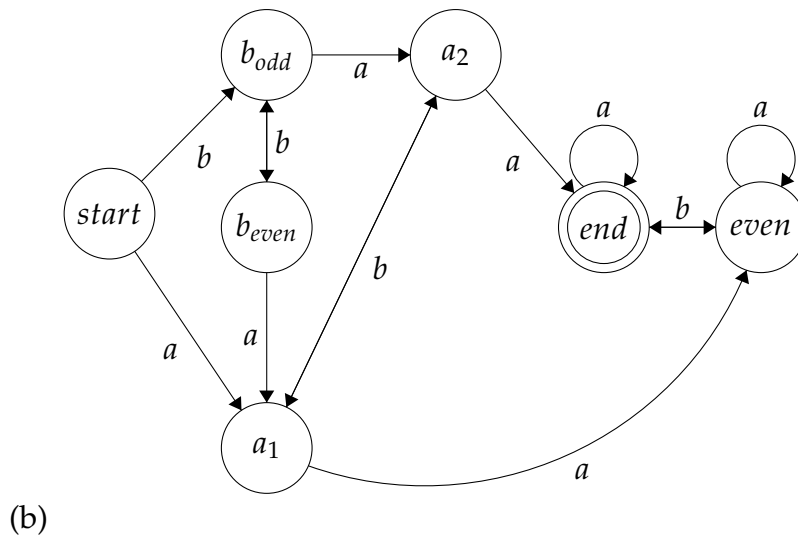
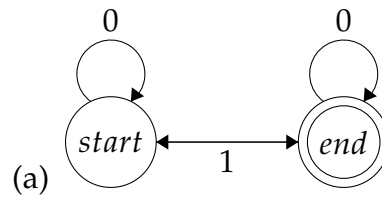
Thus, the change of value of Opt' is ≥ 0 thus Opt' is also optimal.

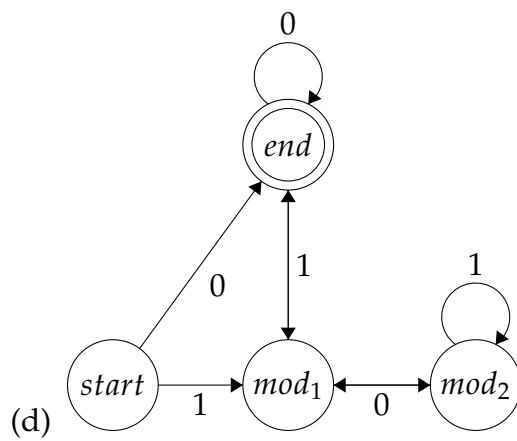
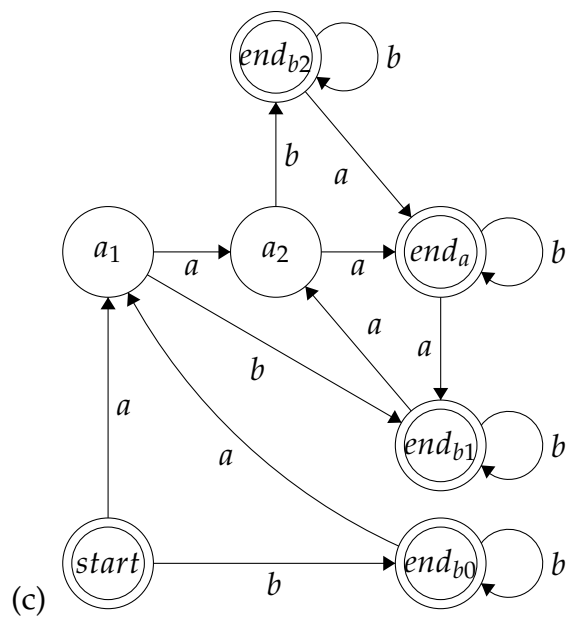
Thus we have produced an optimal solution that agrees with Alg on $1 - j$ items thus we keep repeating this to show that Alg is an optimal solution.

Problem 5

- (a) This is true if the string has at least two a 's anywhere in it. $(a|b)^*a(a|b)^*a(a|b)^*$
- (b) This is true if the string starts with a b has a nonnegative number of consecutive a 's which is divisible by 3 then finished by a b . $b(aaa)^*b$

Problem 6





Problem 7

Problem 8

- (a) Assume $n = 2$ and WLOG let $(a_1 - 1)b_1 \geq (a_2 - 1)/b_2 \implies a_1b_2 - b_2 \geq a_2b_1 - b_1 \implies a_1b_2 - b_2 - a_2b_1 + b_1 \geq 0$

Thus consider $a_1(a_2 + b_2) + b_1 - a_2(a_1 + b_1) - b_2$. This is the difference between applying magic 1 first or magic 2 first.

$$a_1(a_2 + b_2) + b_1 - a_2(a_1 + b_1) - b_2 = a_1b_2 - b_2 - a_2b_1 + b_1 \geq 0$$

Thus since this is a non negative sum we know that applying magic 2 first is better in this situation.

Algorithm 3 MaxMoney

- (b) **Require:** $M = [a_1, b_1] \times [a_2, b_2] \times \dots$ a magic $2 \times n$ matrix of magic s.t. $a_i, b_i \geq 1$.

Ensure: Finds max money you can get from applying the magic.

```

1: function MAXMONEY( $M$ )
2:   if  $|M| = 0$  then
3:     return 1
4:    $money \leftarrow 1$ 
5:    $maxMagic[1, \dots, n] \leftarrow [1, 2, 3, \dots, n]$  ▷ Array of indices
6:    $Sort(maxMagic)$  s.t.  $i < j \iff (a_i - 1)/b_i \leq (a_j - 1)/b_j$  ▷ Sort indices based on
   the value of  $(a_i - 1)/b_i$ , the biggest goes last
7:   for  $i \in \{1, \dots, n\}$  do
8:      $money \leftarrow M[maxMagic[i]][1]money + M[maxMagic[i]][2]$ 
9:   return  $money$ 

```

Let $Alg = \{i_1, \dots, i_n\}$ be the magic we applied at step $j \in \{1, \dots, n\}$. Assume $Opt = \{k_1, \dots, k_n\}$ is an optimal solution that if you apply the magic in the steps it gives it will give you the max money to get. If $Alg = Opt$ then we are done so assume not.

Let j be the first index where Alg and Opt differ.

Thus consider Opt' first set it equal to Opt . Then find where i_j is in Opt then swap it up till you get it in spot j . The reason we can do this is that Alg is greedy we know that $(a_{i_j} - 1)/b_{i_j} \leq (a_{k_l} - 1)/b_{k_l}$ for $l > j$. This implies that

$$a_{k_l}(a_{i_j}m + b_{i_j}) + b_{k_l} - a_{i_j}(a_{k_l}m + b_{k_l})b_{i_j} = (a_{k_l} - 1)b_{i_j} - (a_{i_j} - 1)b_{k_l} \geq 0$$

Where m is the starting money before applying i_j . Thus applying i_j before always

produces at least as much money as if doing k_l first. Thus, we can swap it up till we get to spot j . Thus Opt' is an optimal solution because its money is $\geq Opt$'s money.

Thus, we found an optimal solution that agrees with Alg on $1 - j$ spots, and we can keep doing this till we get an one that agrees on $1 - n$.

Also, this algorithm is $O(n \log n)$ do to the sorting we do on line 6, but then we do a for loop which is $O(n)$, so overall it is $O(n \log n)$.